Cloud Command Parser Project Report

Lightweight Natural-Language Cloud Command Parsing with Distillation and Kubernetes

Detail	Information
Name	Ankit Yadav
Roll number	230101069
Project	Cloud Command Parser
Repository	https://github.com/ankityadav010101/Cloud_Command_Parser

Project overview

Cloud Command Parser turns a short English cloud operation request into a compact, schema-controlled JSON command. The project brings together parallel teacher labeling, validated dataset construction, sequence-level student training, object-backed model delivery, a FastAPI inference service, Kubernetes deployment, and network security exercises. Its design keeps the validation set out of labeling and training, checks model output before returning it, and records reproducible configuration and dataset integrity evidence.

The local pipeline processed all 3,000 supplied training commands. Cleaning retained 2,728 valid unique examples, the NACL simulator passed all six sample decisions, subnet allocations were calculated, and Python source compilation passed.

1. Data pipeline and teacher labeling

`worker/label.py` assigns each input line to a deterministic shard using `line_number % SHARD_COUNT == SHARD_INDEX`. Label workers call the local Ollama `phi3.5` teacher with temperature 0 and seed 42, saving each raw response alongside its source ID and command. The prompt requests the four-field JSON contract. Two workers are configured as a StatefulSet in the `pipeline` namespace.

`worker/clean.py` validates JSON structure, intent membership, and slot text against the original command. It removes duplicate IDs and commands, sorts retained examples by ID, verifies source coverage, writes `datasets/v1/train.jsonl`, and records a SHA-256 checksum. The validation set is reserved for evaluation.

Local cleaning result	Count
Input commands	3,000
Invalid JSON	124
Incorrect key sets	2
Invalid intents	22
Invalid slots	79
Duplicate records	45
Retained training examples	2,728

The cleaned dataset checksum is `5ded5944f8e1d9f3ef3bd5d39e302a5f397465895f80f9d248b39a4d33f029fd`. Counts and checksum are recorded in the project results directory.

2. Student model and sequence-level distillation

The training pipeline configures EleutherAI Pythia-70M as a compact student model. It uses teacher-generated JSON as the target sequence and supports CPU fine-tuning with LoRA, followed by adapter merging for a self-contained serving artifact. `training/config.yaml` centralizes the seed, sample limit, sequence length, batch size, epoch count, learning rate, and weight decay. The code supports separate model versions for rolling deployment exercises.

Sequence-level distillation is suited to the available teacher interface: the student learns to reproduce complete structured responses. Logit-based distillation is outside the implemented pipeline because the teacher interface supplies generated text rather than token probability distributions. The local dataset pipeline completed and retained 2,728 examples. The locally installed Phi-3.5 teacher was evaluated against all 100 held-out validation commands. The run produced 100 predictions with no missing examples. JSON validity was 94%; intent accuracy was 72%, date accuracy 81%, time accuracy 78%, place accuracy 72%, and exact full-frame accuracy 39%. Raw predictions and the score summary are saved in

`results/teacher_predictions.jsonl` and `results/teacher_validation.json`. A trained student artifact was not produced in the local run.

3. Schema-controlled inference service

`service/app.py` implements `POST /parse`, `GET /healthz`, and `GET /readyz`. Successful responses follow the four-key command schema and include model-version and serving-instance headers. Readiness reflects model initialization, and the service loads model and tokenizer files from S3-compatible storage. Kubernetes Secrets provide runtime credentials. The fault-injection option exercises a fail-closed response when generated output is invalid.

This API boundary connects the trained model to the deployment stack while keeping parsing behavior explicit and observable. The service implementation is present in the repository; container execution and load measurements were not produced in the local run.

4. Kubernetes deployment and operations

`k8s/app.yaml` describes the application namespace, two-replica Deployment, rolling update strategy, Service, Ingress, startup/readiness/liveness probes, and resource requests and limits. The Horizontal Pod Autoscaler targets 70% CPU and is configured for one to two replicas. Readiness checks keep traffic away from a pod while its model initializes. The included k3d setup provides a reproducible local cluster configuration.

The manifests define deployment and scaling behavior. Cluster rollout, recovery, and runtime scaling measurements were not produced in the local run. The single-node local cluster configuration is intended for development and does not establish resilience to physical machine failure.

5. Network design and stateless NACL

The `10.20.0.0/16` address block is allocated from largest host requirement to smallest. Usable host counts exclude network and broadcast addresses.

Subnet	Hosts needed	Network	Usable range	Broadcast	Usable hosts
app	1,000	10.20.0.0/22	10.20.0.1 - 10.20.3.254	10.20.3.255	1,022
public	250	10.20.4.0/24	10.20.4.1 - 10.20.4.254	10.20.4.255	254
data	100	10.20.5.0/25	10.20.5.1 - 10.20.5.126	10.20.5.127	126
management	20	10.20.5.128/27	10.20.5.129 - 10.20.5.158	10.20.5.159	30

`network/nacl.json` places the deny rule for `198.51.100.0/24` ahead of broad HTTP/HTTPS allows. Explicit ephemeral-port rules account for stateless return traffic, while SSH access is limited to the management subnet. The supplied packet simulator passed all six expected sample decisions (6/6, 100%). The subnet calculator produced the allocations shown above. Evidence is saved in `results/nacl_sample_check.txt` and `results/subnet_check.txt`.

6. Reproducibility and continuous integration

The repository includes a Makefile, pinned dependency lists, centralized training configuration, unit checks, and GitHub Actions for compilation, testing, evaluation, and secret scanning. Local Python source compilation passed and is recorded in `results/compileall.txt`. The workflow codifies repeatable quality checks for repository changes. A hosted CI result is not claimed here.

7. Performance design, cost model, and review

The project includes the components needed to evaluate the parser as a service: a health and readiness interface, model-version headers, configurable deployment resources, and a load-testing setup. A 99% successful-response objective and a five-second p95 latency target provide concrete service goals for a deployment run.

The supplied hypothetical rate card can estimate hourly replica cost and cost per 1,000 requests once measured traffic is available. This is a planning model rather than a cloud charge. Local latency, throughput, resource consumption, and cloud billing readings were not produced in this run.

Review area	Implemented evidence	Next measurement or improvement
Dataset quality	3,000 inputs cleaned; 2,728 examples retained; checksum recorded	Review label distributions and evaluate on the reserved validation set
Model quality	Teacher: 94% valid JSON and 39% exact full-frame accuracy on 100 held-out commands	Train the student and report exact-frame and field-level validation scores
Service behavior	Schema checks, readiness endpoints, version headers, and fail-closed path	Run container checks and record response/error rates
Performance	Kubernetes resource settings and load-test setup included	Measure p95 latency, throughput, CPU, and memory under stated load
Reliability	Rolling update, probes, and HPA behavior defined in manifests	Exercise rollout, recovery, and scaling in a running cluster
Security and cost	Runtime credentials use Kubernetes Secrets; cost formula documented	Verify secret scanning and calculate cost from measured traffic

8. Limitations

The local run verifies the complete dataset cleaning path, network examples, subnet calculations, source compilation, and the teacher's held-out validation quality. The Pythia student was not trained because the local Python environment does not include the required Transformers and PEFT packages. Container, Kubernetes, and Locust runtime measurements were unavailable because Docker, k3d, kubectl, and Locust are not installed. These measurements require training and running the relevant services in their target environments. The compact student can also inherit teacher labeling errors, and the single-node development cluster cannot demonstrate multi-machine fault tolerance. SeaweedFS supplies local S3-compatible storage for the project workflow; production deployments should use durable, appropriately secured object storage.

9. Conclusion

Cloud Command Parser delivers a coherent end-to-end design for translating natural-language cloud requests into validated structured commands. The implementation connects deterministic parallel labeling and auditable dataset cleaning with a compact-model training path, a schema-controlled API, Kubernetes deployment manifests, and network security exercises. Local evidence confirms that all 3,000 input commands passed through cleaning, yielding 2,728 retained examples, and that the NACL samples, subnet calculator, and Python compilation checks completed successfully. Evaluation of the local teacher on all 100 held-out commands measured 94% JSON validity and 39% exact full-frame accuracy, providing a concrete baseline for student fine-tuning. The repository provides the configuration and operational components needed to train, serve, and evaluate the parser as a cloud-native project.

Sources

• Bastianelli et al., “SLURP: A Spoken Language Understanding Resource Package,” EMNLP 2020: https://aclanthology.org/2020.emnlp-main.588/.

• Kim and Rush, “Sequence-Level Knowledge Distillation,” EMNLP 2016: https://aclanthology.org/D16-1139/.

• EleutherAI Pythia-70M model card: https://huggingface.co/EleutherAI/pythia-70m.

• k3d documentation: https://k3d.io/; Kubernetes documentation: https://kubernetes.io/docs/.